The Word RAM in Lean

A draft section

22 October 2026

This is a draft of one paper section. It presents the formal abstract of the word RAM in Complexitylib the way Demaine and Langerman present the formal abstract of their tiling result [DL26, Section 4]. The Lean code in the boxes is cut by a script from `Complexity/Computability/WordRAM/Abstract.lean` and `AbstractBridge.lean` at commit `939256e6`.

1 The Word RAM in Lean

The word RAM is the usual model for the running time of algorithms on integers. Complexitylib defines it as `Cslib.WordRAM`: its instructions come from a basis that can be changed, and many layers are built on it for writing programs and proving time bounds. These definitions are long. The key point is that a reader does not need to read them to trust a time bound stated with them: it suffices to check that the handful of definitions in a single file, `Abstract.lean`, describe the intended machine, and that two theorem statements in `AbstractBridge.lean` say that it is the library's machine. The Lean proof assistant then verifies the proofs of these statements, and that they rely on only Lean's three standard axioms (`propext`, `Classical.choice`, `Quot.sound`).

Theorem 1.1. *The machine described in this section is the library's word RAM over its standard basis. Every program of either machine has a program of the other, with the same number of registers, such that from every state and at every word size the two take the same step, and halt with the same output after the same number of instructions.*[1] *A problem is decidable in time T on one machine exactly when it is on the other, for the same input encoding, time bound and word-size threshold.*[2]

In the rest of this section, we explain the definitions from `Abstract.lean`. (The source file `Complexity/Computability/WordRAM/Abstract.lean` remains the authoritative version.) It imports only Mathlib and CSLib's base module `Cslib.Init`, and defines the machine from scratch. The two theorems are in a second file, `AbstractBridge.lean`, which imports both `Abstract.lean` and the library's word RAM; its proofs need not be checked by a human.

Lean

```
public import Cslib.Init
public import Mathlib.Data.Nat.Notation
public import Mathlib.Data.Set.Defs
public import Mathlib.Logic.Function.Basic
```

[1] Lean theorem `runsAgree`. See Section 1.7.

[2] Lean theorem `decidableInTime_iff`. See Section 1.7.

1.1 Words and Operations

At word size w, a *word* is a natural number below 2^w. The machine stores arbitrary natural numbers, but an instruction faults when it would create a value that is not a word (Section 1.3).

There are ten binary operations. Subtraction is truncated ($3 - 5 = 0$). Division and remainder round down, with Lean's conventions $x/0 = 0$ and $x \bmod 0 = x$. The operations `and`, `or` and `xor` act bitwise, and `shl` and `shr` shift x left and right by y bits, that is, give $x \cdot 2^y$ and $\lfloor x/2^y \rfloor$. `AluOp.eval` gives the value of an operation on natural numbers, before any check that it is a word.

Lean

```
inductive AluOp
  | add | sub | mul | div | mod | and | or | xor | shl | shr

def AluOp.eval : AluOp → ℕ → ℕ → ℕ
  | .add, x, y => x + y
  | .sub, x, y => x - y
  | .mul, x, y => x * y
  | .div, x, y => x / y
  | .mod, x, y => x % y
  | .and, x, y => x &&& y
  | .or, x, y => x ||| y
  | .xor, x, y => x ^^^ y
  | .shl, x, y => x * 2 ^ y
  | .shr, x, y => x / 2 ^ y
```

1.2 Instructions and Programs

A machine has r registers $R[0], \ldots, R[r-1]$ and a memory cell $M[a]$ at every address $a \in \mathbb{N}$. The number r of registers is a parameter of the types below (the implicit variable `r`). There are ten instructions; the comment on each gives its effect. Register arguments d, i, j, a range over `Fin r` $= \{0, \ldots, r-1\}$ (in `load` and `store`, register a holds the address); a constant x and a jump target t are arbitrary natural numbers. The instruction `width` reads the word size w, so a program can depend on it. A *program* is a finite list of instructions, numbered from 0.

Lean

```
variable {r : ℕ}

inductive Instr (r : ℕ)
  | const (d : Fin r) (x : ℕ)         -- `R[d] := x`
  | copy (d i : Fin r)                -- `R[d] := R[i]`
  | alu (f : AluOp) (d i j : Fin r)   -- `R[d] := f(R[i], R[j])`
  | width (d : Fin r)                 -- `R[d] := w`, the word size
  | load (d a : Fin r)                -- `R[d] := M[R[a]]`
  | store (a i : Fin r)               -- `M[R[a]] := R[i]`
  | jump (t : ℕ)                      -- go to instruction `t`
  | branchEq (i j : Fin r) (t : ℕ)    -- go to `t` if `R[i] = R[j]`, else to the next one
  | branchLt (i j : Fin r) (t : ℕ)    -- go to `t` if `R[i] < R[j]`, else to the next one
  | halt (i : Fin r)                  -- stop with output `R[i]`

abbrev Program (r : ℕ) := List (Instr r)
```

1.3 States and One Step

A *state* consists of the number pc of the next instruction, the registers and the memory. The memory is a function on all of $\mathbb{N}$: there is no allocation and no bound on addresses.

One step of a program continues in a new state, halts with an output, or faults. It executes instruction number pc:

- `const` and `alu` write their value to $R[d]$ if it is a word, and fault otherwise. There is no wrap-around modulo 2^w.
- `width` writes w, which is always a word, since $w < 2^w$.
- `copy`, `load` and `store` move a value without a check.
- `jump` and the two branches set pc; every other instruction except `halt` moves on to instruction pc + 1.
- `halt` i stops with output $R[i]$.
- If pc is not the number of an instruction, that is, pc is at least the length of the program, the step faults.

Copies, loads and stores need no check: when the input fits (Section 1.5), an induction along the run shows that every register and memory cell always holds a word, and so every address used is below 2^w. (This induction is not stated in the abstract files; it only explains why the check is not needed. The library states it for its own word RAM as `Cslib.WordRAM.Runs.bounded_init`.)

Lean

```
structure State (r : ℕ) where
  pc : ℕ
  reg : Fin r → ℕ
  mem : ℕ → ℕ

inductive Outcome (r : ℕ)
  | next (s : State r)
  | halt (v : ℕ)
  | fault

def step (w : ℕ) (p : Program r) (s : State r) : Outcome r :=
  let set (d : Fin r) (x : ℕ) : Outcome r :=  -- `R[d] := x`, then the next instruction
    .next { s with pc := s.pc + 1, reg := Function.update s.reg d x }
  let setWord (d : Fin r) (x : ℕ) : Outcome r := if x < 2 ^ w then set d x else .fault
  match p[s.pc]? with
  | none => .fault
  | some (.const d x) => setWord d x
  | some (.copy d i) => set d (s.reg i)
  | some (.alu f d i j) => setWord d (f.eval (s.reg i) (s.reg j))
  | some (.width d) => set d w  -- `w < 2 ^ w`, so this never faults
  | some (.load d a) => set d (s.mem (s.reg a))
  | some (.store a i) =>
    .next { s with pc := s.pc + 1, mem := Function.update s.mem (s.reg a) (s.reg i) }
  | some (.jump t) => .next { s with pc := t }
  | some (.branchEq i j t) => .next { s with pc := if s.reg i = s.reg j then t else s.pc + 1 }
  | some (.branchLt i j t) => .next { s with pc := if s.reg i < s.reg j then t else s.pc + 1 }
  | some (.halt i) => .halt (s.reg i)
```

1.4 Running Time

`Halts w p s v n` says that from state s, at word size w, program p halts with output v after exactly n instructions, the final `halt` included. It follows `step` one instruction at a time: a fault ends the run without halting, and a run that never halts satisfies `Halts` for no v and n. Every instruction takes one unit of time, whatever its operands and the word size, so n is the running time.

Lean

```
def Halts (w : ℕ) (p : Program r) : State r → ℕ → ℕ → Prop
  | _, _, 0 => False
  | s, v, n + 1 =>
    match step w p s with
    | .next t => Halts w p t v n
    | .halt x => x = v ∧ n = 0
    | .fault => False
```

1.5 Input

An input is a list $x_1, \ldots, x_k$ of natural numbers. The start state is at instruction 0, has all registers 0, and has $k, x_1, \ldots, x_k$ in memory at addresses $0, 1, \ldots, k$ and 0 at every other address; so a program reads the length of its input at $M[0]$. The input *fits* word size w if its length k and all its entries are words. The output of a run is the single number $R[i]$ at its `halt` i.

Lean

```
def init (input : List ℕ) : State r :=
  ⟨0, fun _ => 0, fun a => (input.length :: input).getD a 0⟩

def InputFits (w : ℕ) (input : List ℕ) : Prop :=
  input.length < 2 ^ w ∧ ∀ x ∈ input, x < 2 ^ w
```

1.6 Decision in Time T

A decision problem is a set $A \subseteq \alpha$ of instances, given to the machine by an encoding *encode*: $\alpha \to$ `List` $\mathbb{N}$. Two functions on instances set the terms: $T(a)$ bounds the running time on instance a, and $bits(a)$ is the word-size threshold: the program must work at every word size from $bits(a)$ on that the input fits. A program p *decides* A *in time* T if, for every instance a and every word size $w \geq bits(a)$ that $encode(a)$ fits, p halts within $T(a)$ instructions, with output 1 if $a \in A$ and 0 if not. One program serves every such word size, and can read it with `width`. Word sizes that the input does not fit impose nothing, so the input need not fit at $w = bits(a)$ itself. A is *decidable in time* T if some program, with some number of registers, decides it. (The length condition gives $k < 2^w$, so all input addresses $0, \ldots, k$ fit in a word; for $k > 0$ this is the same as $w > \log_2 k$. `bits` can ask for more.)

Lean

```
def Decides {α : Type*} (p : Program r) (A : Set α) (encode : α → List ℕ) (T bits : α → ℕ) :
    Prop :=
  ∀ a w, bits a ≤ w → InputFits w (encode a) →
    ∃ v n, Halts w p (init (encode a)) v n ∧ n ≤ T a ∧ (a ∈ A → v = 1) ∧ (a ∉ A → v = 0)

def DecidableInTime {α : Type*} (A : Set α) (encode : α → List ℕ) (T bits : α → ℕ) : Prop :=
  ∃ r, ∃ p : Program r, Decides p A encode T bits
```

1.7 Main Theorems

The two theorems are stated in `AbstractBridge.lean`, in the namespace `Cslib.WordRAM.AbstractBridge`. There, names with the prefix `Abstract.` are those of this section, and names without it are from the library's word RAM `Cslib.WordRAM`: `Program .standard r` is a program over the standard basis with r registers, a state $\langle$`pc, ⟨reg, mem⟩`$\rangle$ has the same three parts as here, `step` is one step, and `Executes w q s v n` says that q halts from s with output v after exactly n instructions. A reader does not need to know how these library definitions are made: `decidableInTime_iff` makes the library's decidability in time T equal to that of this section, and `runsAgree` does the same for single steps and halting runs.

Two outcomes, one of each machine, *agree* if both continue in the same state, both halt with the same output, or both fault. An abstract program p and a library program q *run alike* if, from every state and at every word size, one step of each gives agreeing outcomes, and p halts with output v after n instructions exactly when q does.

Lean

```
def OutcomeAgrees : Abstract.Outcome r → Outcome r → Prop
  | .next s, .next t => t = ⟨s.pc, ⟨s.reg, s.mem⟩⟩
  | .halt v, .halt v' => v = v'
  | .fault, .fault => True
  | _, _ => False

def Same (p : Abstract.Program r) (q : Program .standard r) : Prop :=
  ∀ w pc reg mem,
    OutcomeAgrees (Abstract.step w p ⟨pc, reg, mem⟩) (step w q ⟨pc, ⟨reg, mem⟩⟩) ∧
    ∀ v n, Abstract.Halts w p ⟨pc, reg, mem⟩ v n ↔ Executes w q ⟨pc, ⟨reg, mem⟩⟩ v n
```

For every number r of registers, every abstract program runs alike with some library program, and every library program runs alike with some abstract program. (In the proof, the program of the other machine is a translation instruction by instruction.)

Lean

```
theorem runsAgree (r : ℕ) :
    (∀ p : Abstract.Program r, ∃ q : Program .standard r, Same p q) ∧
      ∀ q : Program .standard r, ∃ p : Abstract.Program r, Same p q := ...
```

The two machines have the same time classes: for every type α of instances, problem A, encoding, time bound T and word-size threshold *bits*, A is decidable in time T on the machine of this section exactly when it is on the library's word RAM over the standard basis.

Lean

```
theorem decidableInTime_iff {α : Type u} (A : Set α) (encode : α → List ℕ) (T bits : α → ℕ) :
    Abstract.DecidableInTime A encode T bits ↔
      DecidableInTime .standard A encode T bits := ...
```

Together, these two Lean theorems establish Theorem 1.1. In particular, a library theorem that concludes `DecidableInTime .standard A encode T bits` is, by `decidableInTime_iff`, a time bound on the machine described in this section.

References

[DL26] Erik D. Demaine and Stefan Langerman. Tiling 3D by translates of a single polycube is undecidable. arXiv:2610.12392, 2026. Source: `https://github.com/edemaine/translation-tiling-paper`, commit `5309313e`.